

18.3 SMOlPU: 122.1 μ J/Token Sparse MoE-Based Speculative Decoding Language Processing Unit with Adaptive-Offload NPU-CIM Core

Sangwoo Ha¹, Jingu Lee¹, Youngjin Moon¹, Sunjoo Whang¹, Wooyoung Jo¹, Gwangtae Park¹, Sangjin Kim¹, Soyeon Um², Junha Ryu¹, Yurim Jo¹, Hoi-Jun Yoo¹

¹KAIST, Daejeon, Korea, ²Massachusetts Institute of Technology, Cambridge, MA

Abstract

SMOlPU is an energy-efficient MoE-based speculative decoding LLM processor with an NPU-CIM core. It has 3 features: 1) Token-adaptive expert refinement removes redundant expert activations and schedules expert load order, achieving 2.3 $\times$ /4.2 $\times$ energy efficiency improvement in prefill/decode; 2) An adaptive-offload NPU-CIM core and a top scheduling

unit maintain HW utilization under dynamic INT-FP ratios, achieving 3.3 $\times$ lower latency; 3) A reconfigurable DRAM-based LUT-CIM reduces adder tree power/area, while supporting dynamic input size. The LLM processor achieves 43.5% lower energy per parameter than prior SOTA.

Reasoning strategies such as chain-of-thought (CoT) [1] improve the accuracy of large language models (LLMs) on complex tasks, but lengthen decoding sequences under limited parallelism. The longer sequences repeatedly fetch full model weights from external memory, which inflates external memory access (EMA) and hinders deployment on mobile devices [2-5]. Numerous optimizations have been explored, from conventional methodologies, such as quantization and channel-wise sparsity handling to speculative decoding (SD) [6] and sparse mixture-of-experts (MoE) [7-9] for higher parallelism and reduced EMA. By holistically adopting an MoE-based SD LLM together with the conventional techniques, we can achieve an 89.7% reduction in EMA energy without accuracy loss. However, existing state-of-the-art hardware [2-5] fails to exploit MoE-based SD LLM optimizations. Prior LLM accelerators [2-3] focused on group quantization [10] for low bit-precision and exploiting sparsity for low-power integer (INT) MAC. As group quantization increases the number of floating-point (FP) MACs and exploiting sparsity reduces the number of INT MACs, distinct INT-FP workloads are created. Although NPU with compute-in-memory (NPU-CIM) architectures [11] deliver high energy efficiency on such heterogeneous INT-FP workloads, applying MoE-based SD to NPU-CIM exposes an EMA penalty due to redundancy. As shown in Fig. 18.3.1, the decoding stage suffers from significant weight redundancy, caused by both unnecessary activation of experts for mispredicted (and thus rejected) tokens and sparsity that arises since the expert outputs are scaled by their routing score (RS). In the prefill stage, sequential loading of 4MB experts requires partial sum (PSUM) caching to aggregate all expert outputs, which enlarges the PSUM footprint and further increases EMA. MoE-based SD LLM also induces channel-wise sparsity that varies over time, creating a dynamically changing ratio of INT and FP MACs across quantization groups. While a prior NPU-CIM architecture supports parallel INT-FP processing with fixed INT-FP ratio, MoE-based SD requires INT MAC results to feed into FP MAC operations such as dequantization, FP scaling, and FP aggregation, with varying INT-FP MAC ratios at runtime. Average hardware utilization drops to 33.7-51.1%, which raises end-to-end latency by 3.3 $\times$ even on a state-of-the-art NPU-CIM [11]. Condensing sparse workloads into a dense format in CIM can raise INT MAC utilization of CIM, but demands a reconfigurable adder tree (AT) [12, 13] that supports variable input size, increasing power by 3.1 $\times$ compared with a conventional AT.

To address these issues, we propose SMOlPU, a pipelined integer-floating point NPU-CIM architecture for energy-efficient MoE-based SD LLM inference on mobile devices, with three key features: 1) Token-adaptive Expert Refinement (TaER) with a weight management unit (WMU) and a PSUM management unit (PMU) that eliminates redundant expert fetching and reduces EMA; 2) An adaptive-offload NPU-CIM core (ANC) with a top scheduling unit (TSU) that sustains high utilization and low computing latency under time varying integer to floating-point ratios; 3) A reconfigurable DRAM-based LUT-CIM (RDL-CIM) using bipolar coded DRAM (BCD) that lowers the power overhead of a reconfigurable adder tree. Fig. 18.3.2 shows the overall architecture of SMOlPU, consisting of a memory management unit (MMU), TSU, 4 clusters, and 128KB global memory (GMEM). The MMU comprises WMU for masking redundant experts, and a PMU with 512KB PSUM memory (PSMEM) for eliminating redundant PSUMs. The TSU combines quantization groups into a bundle and forwards indices of bundled groups to the workload allocator of clusters. The cluster consists of 4 ANC, a workload allocator, a scaling and aggregation unit, and 8KB OAMEM. The ANC consists of an index building unit for the CIM LUT, 4 CIM macros with 4 banks, 4 NPUs with a shift-and-accumulate unit (SAC), a 1.5KB WMEM, and an 8KB scaling factor memory (SFMEM). A CIM bank is composed of 32 local cell arrays (LCAs), each containing a 64 $\times$ 4 6T SRAM cell array with a local decoder, two 10 $\times$ 4 3T1C DRAM LUTs, and read word-line (RWL) switches for refresh.

Figure 18.3.3 presents the proposed TaER algorithm integrated into the MMU architecture to reduce EMA from redundant experts and the PSUM. Since only a fraction of candidate tokens (SD outputs) are accepted and each has inter-token correlation, TaER estimates a per-token acceptance rate (AR) that predicts final acceptance and schedules cache management by exploiting expert-overlapped demands across tokens. The MMU realizes TaER with two-stage dataflow composed of the WMU and the PMU. The WMU masks experts and input channels that contribute little to final accepted tokens, reducing EMA and computation energy. The WMU consists of a redundancy detection unit (RDU) for AR estimation and sorting, and a decision and masking unit (DMU) for mask generation. For

expert masking, the RDU aggregates attention scores per token within each MoE layer to compute AR, and the DMU masks experts with AR below a threshold. Within each expert, the RDU sorts input channels by magnitude scaled with the routing score, and the DMU masks channels below a precomputed top-k threshold. Experts masked by the WMU incur neither EMA nor computation. Masked input channels are skipped and channels masked for all routed tokens are not fetched from external memory, reducing expert EMA and computation energy by 66.3% and 80.0%, respectively. The PMU manages the PSUM cache by evaluating the overlap between the set of currently cached tokens and the set of tokens routed to each expert. It sequences expert load order to fetch the expert that is most commonly required by cached tokens to maximize the PSUM reuse and reduce evictions. The PMU operates in two sequential modes: expert sequencing and PSUM ranking. In expert sequencing, a greedy scheduler computes a reuse score for each expert, defined as the number of cached tokens that route to each expert, and loads the expert with the highest reuse score. In PSUM ranking, the PSMEM capacity miss triggers the eviction of the PSUM with the lowest relevance to the currently loaded expert. This maximizes on-chip PSUM reuse, reducing PSUM EMA by 67.2%. Overall, TaER improves energy efficiency by 2.3 $\times$ in prefill and 4.2 $\times$ in decode, while expert-wise pipelined operation keeps latency overhead lower than 3%.

Figure 18.3.4 shows the operation and implementation of the TSU and the ANC, designed to maximize HW utilization under diverse INT-FP ratios. In TaER, fine masking induces sparsity from the RS, which degrades CIM utilization due to finely pruned input channels. Densifying these input channels into bundles improves CIM utilization, but still leads to low NPU utilization when bundles contain fewer quantization groups, leaving FP resources idle during aggregation. We propose an on-the-fly workload allocation scheme that manages CIM-friendly bundles and sparse groups separately, maintaining high utilization on both CIM and NPU. In the TSU, nonzero input channels are fetched and compacted into a group FIFO. A bundling unit then organizes these groups into bundles (up to 4 groups per bundle), which are dispatched to 32 INT MACs and 4 FP MACs, thereby eliminating zero channels. In the ANC, sparse groups are collected into a leftover FIFO and offloaded to the NPU whenever FP resources are available. The hardware autonomously monitors FP MAC availability, enabling offloading without software intervention. For example, when a bundle contains 4 groups, the CIM and NPU operate in a fully pipelined manner. The CIM first performs bit-parallel bit-serial (BPBS) INT MACs, and the outputs are then forwarded to NPU, where the SAC computes a weighted sum before FP MAC. 4 SACs are time-division multiplexed onto a single FP MAC, fully hiding 4-cycle weighted-sum latency. When a bundle has fewer than four groups, leftovers are fed to an underutilized NPU to fill idle FP slots. In this case, the SAC performs INT MACs, and forwards results to FP MAC. This is enabled by SAC reconfigurability, achieved using simple AND gates at its input, which supports INT MAC execution in the NPU with negligible area overhead. Overall, the proposed TSU and ANC enhance HW utilization for INT and FP MAC by 23.1% and 35.2%, respectively, achieving a 3.3 $\times$ end-to-end execution speedup.

Figure 18.3.5 depicts the RDL-CIM with BCD which reduces the power consumption of the reconfigurable AT. Low-power CIMs [14, 15] adopt a LUT-based AT that replaces multi-level gate switching with a small number of memory read operations. The proposed RDL-CIM utilizes a reconfigurable LUT decoder to support low-power LUT for diverse input sizes. Furthermore, BCD enables the RDL-CIM to overcome the significant area overhead of the LUT. The RDL-CIM reads 4 weights per LCA and supplies them to the local decoder, which activates a single bit-line of the LUT. The entry on the selected bit-line is read out from 3T1C DRAM cells to RWL. While a standard LUT requires a 4b input to generate 16 entries, BCD makes it possible using only 8 entries with bipolar coding. Even though bipolar coding incurs overhead for inverters and MUXes, smaller entries and shared front-end inverters/MUXes across CIM macros make the logic overhead negligible. Physical area is also reduced by using a compact 3T1C DRAM cell for the LUT. Consequently, LUT pitch is reduced by 75.7% and matches precisely the 3.6 μ m pitch of the SRAM LCA. The local decoder with a router and two 2-to-4 decoders enables reconfigurable LUT-based addition which has 3 different modes: one group, symmetric two group, and asymmetric two group. For example, one group mode decodes W3 and W4 inputs into a single 3-to-8 together with W2 output gating. In the case of asymmetric two group, W3 and W4 inputs decode into 2-to-4 and W1 decodes as 1-to-2. Since any bundle created by the TSU has a maximum of

two groups per 4 weights, all workloads are fully covered by the RDL-CIM. The proposed RDL-CIM reduces power and area by 36.8% and 26.3%, respectively, with only 0.13% refresh power overhead.

Figure 18.3.6 shows measurement results of SMOlPU and a comparison table. SMOlPU is evaluated on MT-bench with GPT-4 as judge (LLM-as-a-judge) [16] using OLMoE [8], DeepSeek-V2 [7], and Qwen3 [9] models. For a sequence of 128 input and 64 output tokens, SMOlPU achieves 34.7-99.3mJ/token with negligible accuracy loss (<0.2), yielding a 42.7-71.1% reduction in total energy consumption. Previous works [2-5] applied various compression techniques to reduce EMA, but were fundamentally limited by degrading model accuracy (3.3 ppl in [3]). In contrast, SMOlPU activates only 1.4B of 15.7B parameters per token, yielding 11.2x EMA reduction, the highest among prior works. SMOlPU also achieves the lowest energy per token per parameter of 7.8μJ/token/parameter, which is 43.5% lower than the state-of-the-art [2] for 1024 input tokens and 1 output token. Its improvement becomes larger with longer output tokens, making the hardware effective for reasoning tasks. SMOlPU is fabricated in 28nm CMOS technology and occupies a 20.25mm² die area, as shown in Fig. 18.3.7. In conclusion, SMOlPU demonstrates a 122.1μJ/token sparse MoE-based speculative decoding LLM inference with a pipelined NPU-CIM architecture for mobile devices.

References:

- [1] Wang, Xuezhi et al., "Self-Consistency Improves Chain of Thought Reasoning in Language Models," arXiv preprint arXiv:2203.11171, 2022. <https://doi.org/10.48550/arXiv.2203.11171>
- [2] S. Kim et al., "Slim-Llama: A 4.69mW Large-Language-Model Processor with Binary/Ternary Weights for Billion-Parameter Llama Model," *ISSCC*, pp. 421-423, 2025. <https://doi.org/10.1109/ISSCC49661.2025.10904761>
- [3] W. Jo et al., "BROCA: A 52.4-to-559.2mW Mobile Social Agent System-on-Chip with Adaptive Bit-Truncate Unit and Acoustic-Cluster Bit Grouping," *ISSCC*, pp. 418-420, 2025. <https://doi.org/10.1109/ISSCC49661.2025.10904658>
- [4] Y. Qin et al., "An 88.36TOPS/W Bit-Level-Weight-Compressed Large-Language-Model Accelerator with Cluster-Aligned INT-FP-GEMM and Bi-Dimensional Workflow Reformulation," *ISSCC*, pp. 420-422, 2025. <https://doi.org/10.1109/ISSCC49661.2025.10904774>
- [5] S. Kim et al., "C-Transformer: A 2.6-18.1μJ/Token Homogeneous DNN-Transformer/Spiking-Transformer Processor with Big-Little Network and Implicit Weight Generation for Large Language Models," *ISSCC*, pp. 368-370, 2024. <https://doi.org/10.1109/ISSCC49657.2024.10454330>
- [6] Li, Yuhui et al., "EAGLE-3: Scaling Up Inference Acceleration of Large Language Models via Training-Time Test," arXiv preprint arXiv:2503.01840, 2025. <https://doi.org/10.48550/arXiv.2503.01840>
- [7] Liu, Aixin et al., "DeepSeek-V2: A Strong, Economical, and Efficient Mixture-Of-Experts Language Model," arXiv preprint arXiv:2405.04434, 2024. <https://doi.org/10.48550/arXiv.2405.04434>
- [8] Muennighoff, Niklas et al., "OLMoE: Open Mixture-Of-Experts Language Models," arXiv preprint arXiv:2409.02060, 2024. <https://doi.org/10.48550/arXiv.2409.02060>
- [9] Yang, An et al., "Qwen3 Technical Report," arXiv preprint arXiv:2505.09388, 2025. <https://doi.org/10.48550/arXiv.2505.09388>

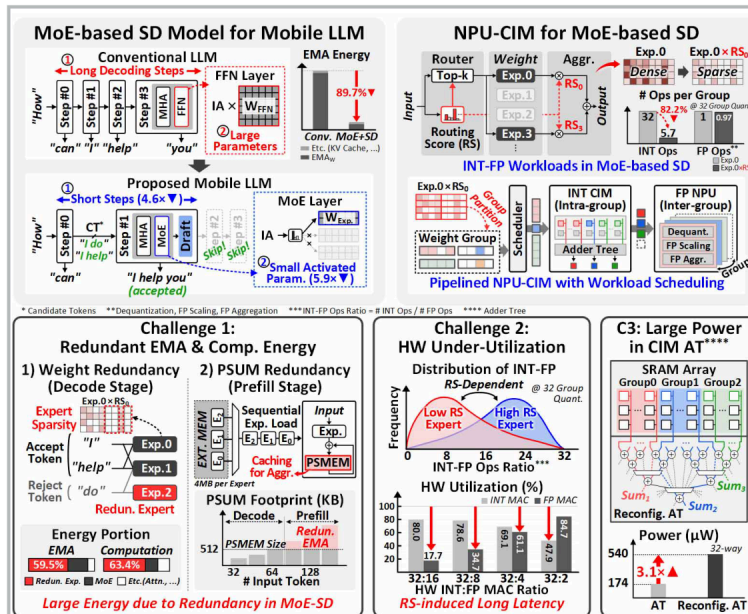


Figure 18.3.1: MoE-based speculative decoding LLM and its design challenges.

- [10] Lin, Ji et al., "AWQ: Activation-Aware Weight Quantization for On-Device LLM Compression and Acceleration," *MLSys*, pp. 87-100, 2024. <https://doi.org/10.48550/arXiv.2306.00978>
- [11] J. Yue et al., "A 28nm 16.9-300TOPS/W Computing-in-Memory Processor Supporting Floating-Point NN Inference/Training with Intensive-CIM Sparse-Digital Architecture," *ISSCC*, pp. 252-253, 2023. <https://doi.org/10.1109/ISSCC42615.2023.10067779>
- [12] Z. Wu et al., "CELLA: A 28nm Compute-Memory Co-Optimized Real-Time Digital CIM-Based Edge LLM Accelerator with 1.78ms-Response in Prefill and 31.32 Token/s in Decoding," *IEEE Symp. VLSI Circuits*, 2025. <https://doi.org/10.23919/VLSITechnologyandCirc65189.2025.11075101>
- [13] R. Guo et al., "CIMFormer: A Systolic CIM-Array-Based Transformer Accelerator With Token-Pruning-Aware Attention Reformulating and Principal Possibility Gathering," *IEEE JSSC*, vol. 59, no. 10, pp. 3317-3329, Oct. 2024. <https://doi.org/10.1109/JSSC.2024.3402174>
- [14] Y. Wang et al., "LLM-CIM: A 28nm 126.7TOPS/W Input-LUT-Based Digital CIM Macro with Reconfigurable Matrix Multiplication and Nonlinear Operation Modes for LLMs," *IEEE Symp. VLSI Circuits*, 2025. <https://doi.org/10.23919/VLSITechnologyandCirc65189.2025.11074939>
- [15] Y. He et al., "A 28nm 2.4Mb/mm² 6.9 - 16.3TOPS/mm² eDRAM-LUT-Based Digital Computing-in-Memory Macro with In-Memory Encoding and Refreshing," *ISSCC*, pp. 578-580, 2024. <https://doi.org/10.1109/ISSCC49657.2024.10454323>
- [16] Zheng, Lianmin et al., "Judging LLM-as-a-Judge with MT-bench and Chatbot Arena," *NeurIPS*, pp. 46595-46623, 2023. <https://doi.org/10.48550/arXiv.2306.05685>

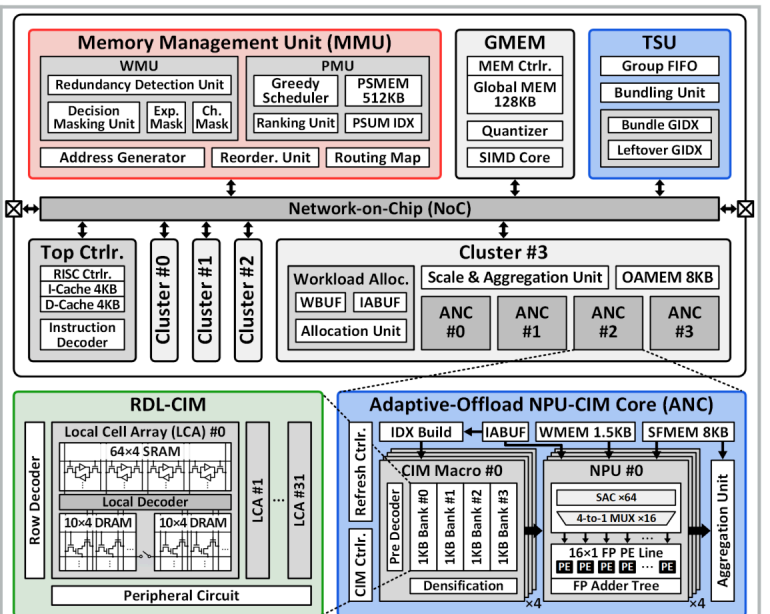


Figure 18.3.2: Overall architecture.

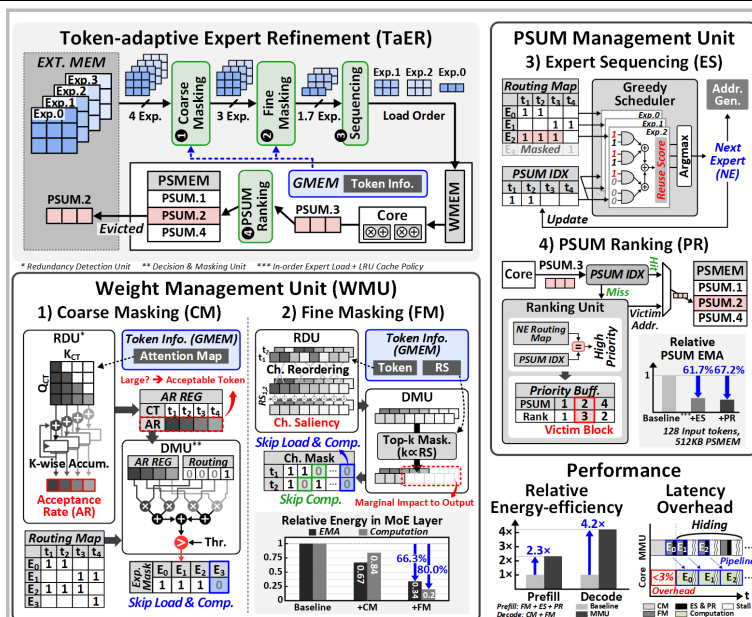


Figure 18.3.3: Operation of TaER and implementation of the MMU.

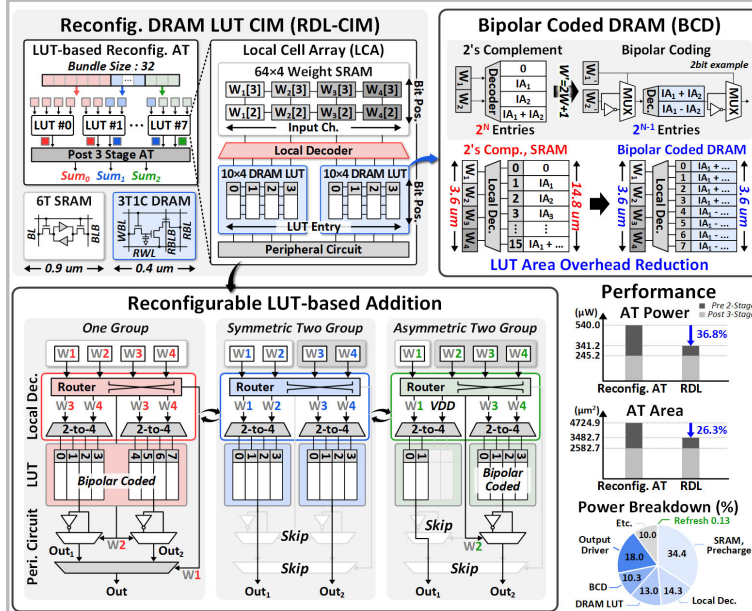


Figure 18.3.5: The RDL-CIM with BCD and operation flow of the reconfigurable LUT.

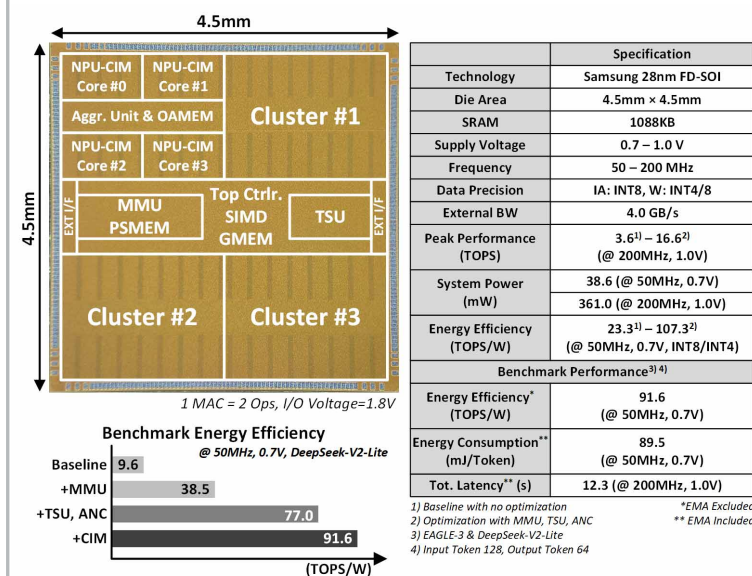


Figure 18.3.7: Chip micrograph and performance summary.

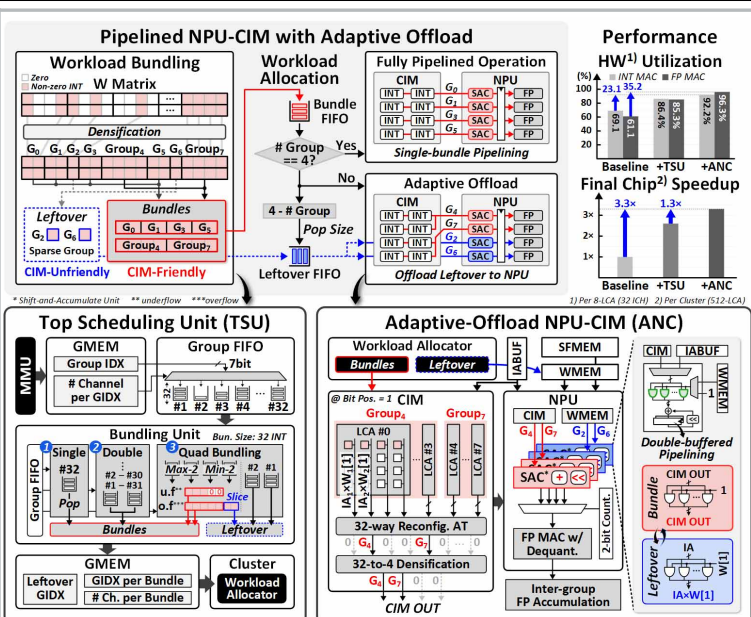


Figure 18.3.4: Operation and implementation of the TSU and ANC.

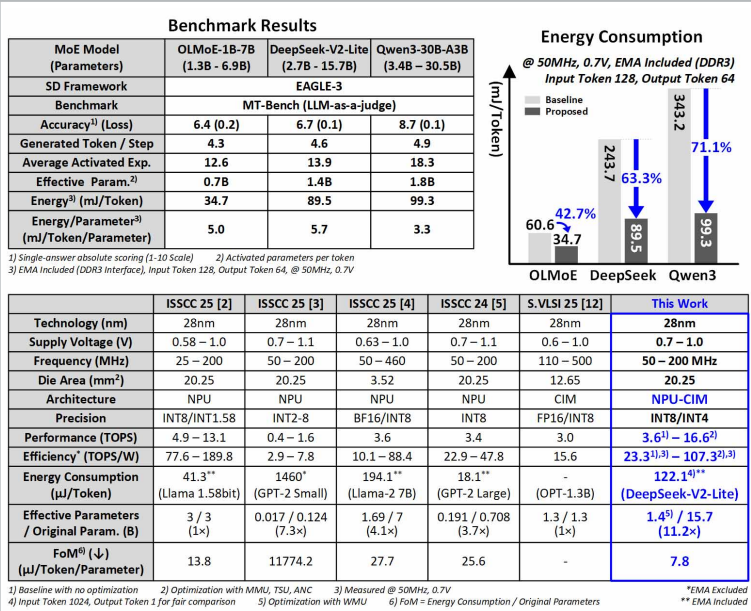


Figure 18.3.6: Measurement results and performance comparison table.